



جامعة أبوظبي
Abu Dhabi University

Department of Computer Science
and Information Technology

Programming Assignment 3

CSC408: Distributed Information Systems

**Implementing a Secure, Scalable
Emergency Coordination Platform –
EmergencyConnectUAE™**



**Emergency
ConnectUAE**

Objective

In alignment with the UAE's strategic focus on **public safety, national resilience, and smart government services**, this programming assignment focuses on designing and implementing **EmergencyConnectUAE™**, a **distributed emergency coordination platform** that enables secure, reliable, and scalable coordination between **emergency responders, hospitals, and control centers**.

EmergencyConnectUAE™ aims to support **time-critical emergency operations** such as incident reporting, emergency unit dispatching, resource reservation, and real-time status tracking, while ensuring **strong security, concurrency control, and system resilience** under high load.

Through this assignment, students will apply advanced concepts in **distributed information systems**, with particular emphasis on **Redis-based mechanisms, secure RESTful services, and real-world system design choices**.

Assignment Description

You are required to **design and implement a secure RESTful API-based distributed system using Spring Boot**, supported by **Redis**, that powers the EmergencyConnectUAE™ platform.

The system must:

- Handle **multiple concurrent emergency incidents**
- Coordinate **shared and limited resources** (e.g., ambulances, emergency units)
- Remain **responsive and correct under high load**
- Enforce **strong security controls** suitable for a public-safety system

Your implementation must reflect **real-world emergency scenarios**, not hypothetical or toy use cases.

Core Functional Requirements

1. **API Design:** Design a RESTful API that strictly adheres to **REST principles** and best practices:
 - Stateless endpoints
 - Clear resource naming
 - Consistent HTTP methods and response codes
2. **Emergency Operations Endpoints:** Your system must provide endpoints for:

- Emergency incident reporting
- Incident status updates (OPEN, IN_PROGRESS, RESOLVED)
- Assignment of emergency units (ambulances, responders, hospitals)
- Resource availability queries
- Incident history and audit logs
- Secure access for different system roles

3. Data Model: Your data model must represent:

- Incidents
- Emergency units and resources
- Assignments and reservations
- User roles (dispatcher, responder, hospital admin)
- Authentication and session data
- Event and access logs

Data models must support:

- High concurrency
- Data integrity
- Horizontal scalability

Mandatory Non-Functional Requirements

4. REST API Pagination: All REST APIs that return collections (lists of data) must implement pagination.

- Pagination must be implemented using standard query parameters such as:
 - page
 - size (or limit)
- APIs must return:
 - Paginated results (not full datasets)
 - Metadata including:
 - current page
 - page size
 - total number of records (optional but recommended)

5. Redis Usage: Students **must use Redis** in the following three roles. Failure to demonstrate correct usage will result in major grade penalties.

5.1 Redis Caching: Redis must be used to cache:

- Active incidents
- Resource availability
- Incident summaries for dashboards

Caching must:

- Reduce database load
- Improve response latency
- Handle cache invalidation correctly

5.2. Redis Distributed Locks: Redis distributed locks **must be used** to protect critical operations, such as:

- Assigning an emergency unit to an incident
- Reserving limited medical resources

The system must clearly demonstrate that **without distributed locks, race conditions occur**, and **with locks, correctness is preserved**.

5.3. Redis Session Storage: Redis must be used to store:

- User session data
- Authentication state
- Role and access context

The system must support:

- Stateless backend services
- Horizontal scaling
- Session persistence across server restarts

6. Security: Your system must be secured using the following mechanisms:

6.1. Rate Limiting: Limit the number of requests per user/IP, and Protect the system against abuse and denial-of-service scenarios

6.2. Data Protection – SSL/TLS: All communication must be secured using HTTPS, and Sensitive data must never be transmitted in plaintext.

6.2. JWT Authentication (Token-Based): Authenticate users using JWT, **and** Tokens must include role and authorization claims.

6.3. Access Resources with JWT: Protect all sensitive endpoints and Enforce role-based access control using JWT claims.

6.4. Input Validation: Validate all incoming data, **and** Protect against injection and malformed requests

6.5. Multi-Factor Authentication (MFA): Implement MFA for at least one critical user role (e.g., dispatcher or administrator)

5.6. Prevent Directory Traversal Attacks: Ensure no endpoint can be used to access unauthorized filesystem locations

6.7. Blacklist & Whitelist Enforcement: Implement blacklists or whitelists where appropriate (e.g., IPs, roles, operations)

6.8. CORS Configuration: Properly configure Cross-Origin Resource Sharing rules. Prevent unauthorized cross-domain access.

6.9. Secure Application Properties: Protect sensitive configuration values. No secrets or credentials should be hard-coded.

6.10. Enforce HTTP Security Headers: Your application must enforce:

- X-Content-Type-Options
- X-Frame-Options
- Content-Security-Policy
- Strict-Transport-Security
- Other relevant HTTP security headers

6. Innovation (Bonus – up to 2.5 marks): The inclusion of advanced functionalities will lead to higher grades. Bonus marks, up to 2.5 marks, may be awarded at the instructor's

discretion based on the overall creativity and effectiveness of the advanced solution.
Creativity and real-world relevance are strongly encouraged.

7. **RestAPI Documentation:** Provide a **comprehensive RESTful API documentation produced using Swagger**, presenting clear endpoint descriptions, request/response schemas, authentication mechanisms, and practical usage examples.

If you have any questions or need clarification on any aspect of the assignment, feel free to ask. Best of luck, and let's work together to build a trusted, family-centric community that fosters support, solidarity, and meaningful connections in line with the UAE's Year of Family initiative!!!

Deliverables

1. **Full project with all Java files** of the developed system. **All files must include Section number, Group number, Student IDs and names of all members. Failure to do that will result in 1-mark penalty.**
2. **Video:** Recorded demonstration showcasing the Emergency Coordination Platform and its features.
3. **Report:** A detailed report that includes:
 - a) **System architecture and flow description, and database ERD.**
 - b) **A comprehensive RESTful API documentation produced using Swagger**, presenting clear endpoint descriptions, request/response schemas, authentication mechanisms, and practical usage examples.
 - c) **Test cases and stress testing results:** Test cases and results demonstrating the functionality and reliability of the API, including security testing scenarios, and concurrency simulations.
 - d) **Self-evaluation and future improvements:** Self-evaluation of the implemented solution, including **critiques**, and suggestions for potential improvements.
 - e) **Dedicated section (VERY IMPORTANT):** Your report **must include a dedicated section** discussing the following **three Redis mechanisms**:
 - **Redis Caching**
 - How it is used in your system
 - Real emergency-related use cases
 - Benefits (performance, scalability, resilience)
 - **When Redis caching should NOT be used**
 - **Redis Distributed Locks**
 - Which operations require locking
 - Real-world race condition scenarios
 - Benefits (correctness, consistency)
 - **When distributed locks should NOT be used**
 - **Redis Session Storage**
 - Why sessions are stored in Redis
 - Benefits in distributed systems
 - Fault tolerance and scalability advantages
 - **When session storage in Redis is NOT appropriate**

This discussion must be **analytical and technical**, not descriptive.



Due Date

May 31st, 2026, at 23:59. Please note that this deadline is strictly enforced, and late submissions will incur a penalty of -5% per day. **Students can work in groups of at most three students.**

Evaluation:

Programming Assignment 3 will be evaluated based **solely on a DEMO and using the below rubric**. Every group member is required to conduct an individual demonstration of their work and showcase his contributions and proficiency in the developed system. If a student or group fails to conduct a demo or is absent on the designated demo day, they will receive a score of **ZERO** for this assignment.

This assignment is worth **5 marks with EXTRA 2.5 BONUS** towards the final grade of this course. **Demo Days:** All demos will be scheduled during the period June 1-4, 2026.

Evaluation Rubric

The below evaluation rubric will be used during the demo, and failure to demonstrate the required functionality and meet the criteria outlined in this rubric will result in a grade of zero for the Programming Assignment 3. It is essential that you prepare and present a comprehensive demo to showcase your understanding and implementation of the requested tasks.

Core Assessment Rubric (5 Marks)

Category	Description (Updated to Match Assignment 3)	Max Marks
1. RESTful API Design & Core Emergency Operations	Correct and complete implementation of RESTful APIs following REST principles (statelessness, resource orientation, proper HTTP verbs and status codes). Includes emergency incident reporting, status transitions, unit assignment, resource queries, audit/history access, and role-based protected endpoints reflecting realistic emergency workflows.	1.0
2. Frontend Application (Usability & Integration)	Functional frontend demonstrating key EmergencyConnectUAE™ workflows (incident creation, status tracking, unit assignment, dashboards). Clear integration with backend APIs, secure authentication flow, and meaningful visualization of incident and resource state (no mock-only interfaces).	0.5
3. Security, Authorization, Concurrency & Scalability Mechanisms	Correct implementation of <i>mandatory</i> security and distributed system mechanisms, including: HTTPS, JWT authentication and RBAC, MFA for at least one critical role, rate limiting, Redis-backed session storage, input validation, CORS, HTTP security headers, secure configuration handling, blacklist/whitelist enforcement, and explicit prevention of race conditions using Redis distributed locks. Design choices must support horizontal scalability and high concurrency.	1.25
4. Redis Usage & System Correctness Under Load	Effective and clearly demonstrated use of Redis for all three required roles: caching (with proper invalidation), distributed locking for critical sections (e.g., resource assignment), and session storage. Includes validation or simulation that demonstrates correct behavior under concurrent incidents and competing resource requests.	1.25
5. Testing, Validation & Simulation	Quality of testing and validation efforts, including concurrency testing (multiple incident submissions or unit assignments), correctness verification under load, and handling of failure or contention scenarios. Evidence may include logs, test cases, screenshots, or short simulations.	0.5
6. Documentation, Presentation & Self-Evaluation	Clear and thorough technical documentation including Swagger/OpenAPI documentation, configuration explanation, Redis role justification, and security overview. Includes a concise recorded demo video. Also includes a self-evaluation discussing design tradeoffs, limitations, and concrete suggestions for future improvements.	0.5

Total Core Marks: 5.0

BONUS: Innovation & Advanced Features (up to 2.5 Marks)

Bonus marks are awarded **only after satisfying all mandatory requirements**.

Examples of bonus-worthy contributions include (but are not limited to):

Innovation Area	Examples
Advanced Distributed Design	Priority-based incident dispatching, geo-aware or region-based sharding, failover strategies, or leader-election logic

Innovation Area	Examples
Advanced Security	Adaptive rate limiting, security analytics dashboards, anomaly detection, or advanced audit logging
Operational Excellence	Monitoring/metrics, health checks, circuit breakers, graceful degradation strategies
Real-World Enhancements	Simulation of hospital capacity constraints, emergency escalation logic, SLA-aware dispatching, or realistic UAE-centric workflows
Exceptional Engineering Quality	Clean architecture, modular design, strong separation of concerns, or performance optimizations backed by evidence

Maximum Bonus: 2.5 marks

Important Notes

- Failure to correctly demonstrate **all three mandatory Redis roles (cache, locks, sessions)** will result in **significant grade penalties**.
- Emphasis is on **realistic, production-style system design**, not toy examples.